

Activity 07: Understanding and Reducing AI Hallucinations

Sharie Rhea

Southern New Hampshire University

CS 530: Artificial Intelligence Principles and Applications

Joshua Andrews

March 22, 2026

To explore code hallucinations I landed on the idea of querying song metadata such as title, artist, album, duration, listeners, play count, and tags from last.fm. The initial prompt I gave to ChatGPT was “Simple python script that displays song details from last.fm.” I chose this vague prompt specifically because from my research it seems that AI tools often hallucinate nonexistent libraries or application programmer interface (API) endpoints (Pavel et al. 2025). In this case, ChatGPT was able to successfully create a short Python script using the last.fm API and the requests library. It used the exact endpoint I had in mind, `track.getInfo`, and even included error handling for when a data element is missing. However, the script included one critical hallucination: `API_KEY = "YOUR_API_KEY" # replace with your Last.fm API key.` This line of code encourages the user to place their sensitive API key directly into the code. This security vulnerability is an example of a code quality hallucination (Pavel et al. 2025).

There are two main contributing causes for this particular hallucination. Unfortunately, ChatGPT is likely replicating numerous last.fm API applications or examples available on the web that use this structure. This method of placing keys or tokens directly into source code is simple and effective, configuring a more secure method of authentication requires thought and effort that tutorials or examples avoid. If the programmer is not careful, hardcoded keys may be committed to version control and inadvertently leaked, resulting in compromised API accounts that may be impossible to recover. The other contributing cause for this hallucination was the vague initial prompt. If I had included instructions to be security conscious or mentioned that this application would handle sensitive user data and keys, ChatGPT would have been more likely to draw from robust code patterns. To resolve this hallucination, I simply pointed out the security flaw to ChatGPT, which then suggested using an environment variable or the Python `dotenv` library with a configuration file.

ChatGPT suggested expanding the project into a command line interface (CLI), which I agreed to. After reviewing and testing the subsequent code I discovered two additional hallucinations. ChatGPT mentioned an “artist <name>” command that would return statistics about the given artist, but no code was generated to support this functionality. I would categorize this hallucination as a functional correctness issue (Pavel et al., 2025). This hallucination may have again been a result of a vague prompt or undefined requirements. Additionally, ChatGPT was attempting to refactor the code drastically, and it is possible the context was too complex to track each desired feature. The impact of not identifying hallucinations like these can be great. Without robust unit and integration testing, small feature details that are not frequently used may slip by unnoticed for long periods of time. The final hallucination I encountered was another code quality issue. The script and CLI incorrectly handled song titles and artists containing more than one word. Improper input validation is another common location for bugs and vulnerabilities. In this case, the code generated by ChatGPT did not correctly parse quotation pairs, and could only return results when the artist’s name was a single word. Once again, I was able to resolve both of these issues by pointing out their existence to ChatGPT.

Current practice for reducing hallucinations in AI-generated code calls for providing enough relevant detail and context in the prompt. In particular, Pavel et al. (2025) found that including API or library details along with a code template or framework led to the greatest reduction in hallucinations. In addition, prompt iteration and refinement focused on code security, simplicity, and structure can help mitigate the vulnerabilities, code “smell,” and overly complicated logic that AI-generated code can produce. Code must also be properly reviewed by humans to ensure it meets quality and safety standards before use in real-world applications.

References

OpenAI. (2026). *ChatGPT* (GPT-5.2). <https://chat.openai.com/chat>

Faker, O., & Toker, S. (2026). Informed Skepticism: Navigating The Code Hallucination Dilemma. *2026 5th International Informatics and Software Engineering Conference (IISEC)*, Page 634-Page 639. <https://doi.org/10.1109/iisec69317.2026.11418436>

Pavel, M., Petrovic, N., Mazur, L., Zolfaghari, V., Pan, F., & Knoll, A. (2025). Hallucination in LLM-Based Code Generation: An Automotive Case Study. *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, 101–106. <https://doi.org/10.1109/fllm67465.2025.11391125>